



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ** | **Факультет прикладной
математики и информатики**

Кафедра теоретической и прикладной информатики
Лабораторная работа № 3
по дисциплине «Методы оптимизации»

МЕТОД ШТРАФНЫХ ФУНКЦИЙ

Бригада 4

ПМ-34 РУБЦОВ АРТЁМ

ПМ-34 КУДЮКОВ МАКСИМ

Преподаватель

ЛЕМЕШКО БОРИС ЮРЬЕВИЧ

Новосибирск, 2026

Содержание	
1 Цель работы.....	3
2 Задание.....	3
3 Исследования	4
3.1 Метод штрафных функций задача (а).....	4
3.1.1 Зависимость от выбора штрафных функции.....	4
3.1.2 Зависимость от g_0	5
3.1.3 Зависимость от стратегии изменения g	6
3.1.4 Зависимость от начальной точки	7
3.1.5 Зависимость от точности ϵ	8
3.2 Метод штрафных функций задача (б)	9
3.2.1 Зависимость от выбора штрафных функции.....	9
3.2.2 Зависимость от g_0	10
3.2.3 Зависимость от стратегии изменения g	11
3.2.4 Зависимость от начальной точки	12
3.2.5 Зависимость от точности ϵ	13
3.3 Метод барьерных функций (а)	14
3.3.1 Зависимость от выбора барьерных функции	14
3.3.2 Зависимость от g_0	15
3.3.3 Зависимость от стратегии изменения g	16
3.3.4 Зависимость от начального приближения.....	17
3.3.5 Зависимость от точности ϵ	18
4 Вывод	19
5 Программа	19
5.1 Main.cpp	19
5.2 PenaltyBarrierMethod.cpp.....	24
5.3 PenaltyBarrierMethod.h	29
5.4 Gauss.cpp.....	31
5.5 Gauss.h	36
5.6 Func.cpp	37
5.7 Func.h	41
5.8 VecOperations.cpp	41
5.9 VecOperations.h	42

1 Цель работы

Ознакомиться с методами штрафных функций при решении задач нелинейного программирования. Изучить типы штрафных и барьерных функций, их особенности, способы и области применения, влияние штрафных функций на сходимость алгоритмов, зависимость точности решения задачи нелинейного программирования от величины коэффициента штрафа.

2 Задание

№	Вид работы
1	Применяя методы поиска минимума 0-го порядка, реализовать программу для решения задачи нелинейного программирования с использованием метода штрафных функций.
2	Исследовать сходимость метода штрафных функций в зависимости от – выбора штрафных функций, – начальной величины коэффициента штрафа, – стратегии изменения коэффициента штрафа, – начальной точки, – задаваемой точности ϵ . Сформулировать выводы.
3	Применяя методы поиска минимума 0-го порядка, реализовать программу для решения задачи нелинейного программирования с ограничением типа неравенства (только задача а) с использованием метода барьерных функций.
4	Исследовать сходимость метода барьерных функций (только задача а) в зависимости от – выбора барьерных функций, – начальной величины коэффициента штрафа, – стратегии изменения коэффициента штрафа, – начального приближения, – задаваемой точности ϵ . Сформулировать выводы

4 Вариант

№	Первая задача (а)	Вторая задача (б)
4	$f(x, y) = 2(y-x)^2 + 14(y-3)^2 \rightarrow \min$ $y - x \geq 1$	$f(x, y) = 2(y-x)^2 + 14(y-3)^2 \rightarrow \min$ $x = -y$

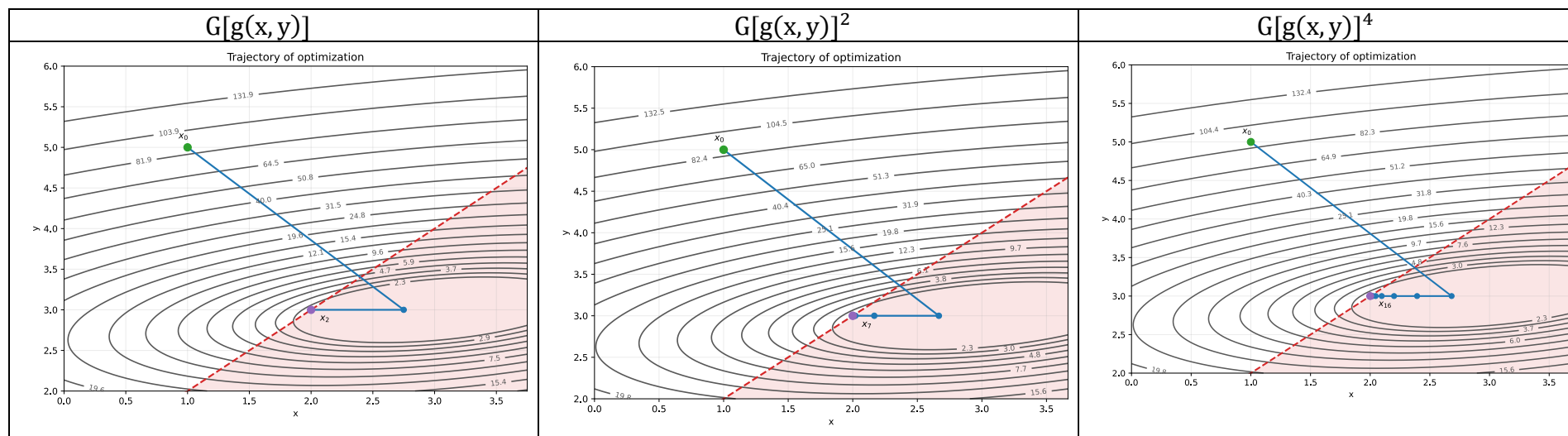
3 Исследования

3.1 Метод штрафных функций задача (а)

3.1.1 Зависимость от выбора штрафных функции

$$G[g(x, y)] = \max\{0, g(x, y)\}$$

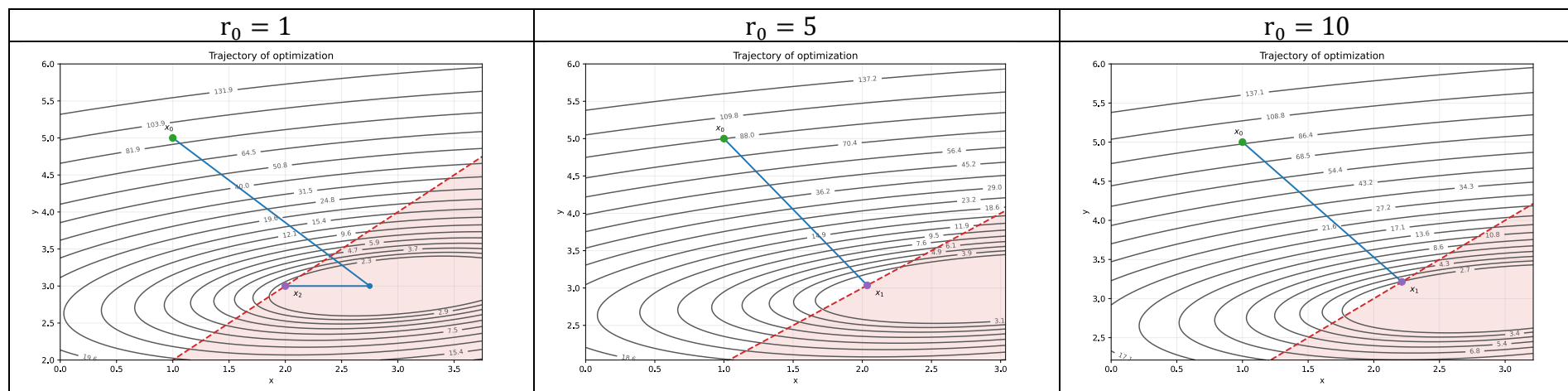
Штрафные ф-и	(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
$G[g(x, y)]$	(1;5)	1e-5	1	$r_k = 10r_{k-1}$	2	822	1.999997933	3.00E+00	2
$G[g(x, y)]^2$					7	1949	1.999997038	2.999994	1.999988
$G[g(x, y)]^4$					10	3698	2.000015949	3.000006	1.999962



Линейная штрафная функция требует меньших вычислений, при повышении порядка штрафной функции “удорожается” расчёт.

3.1.2 Зависимость от r_0

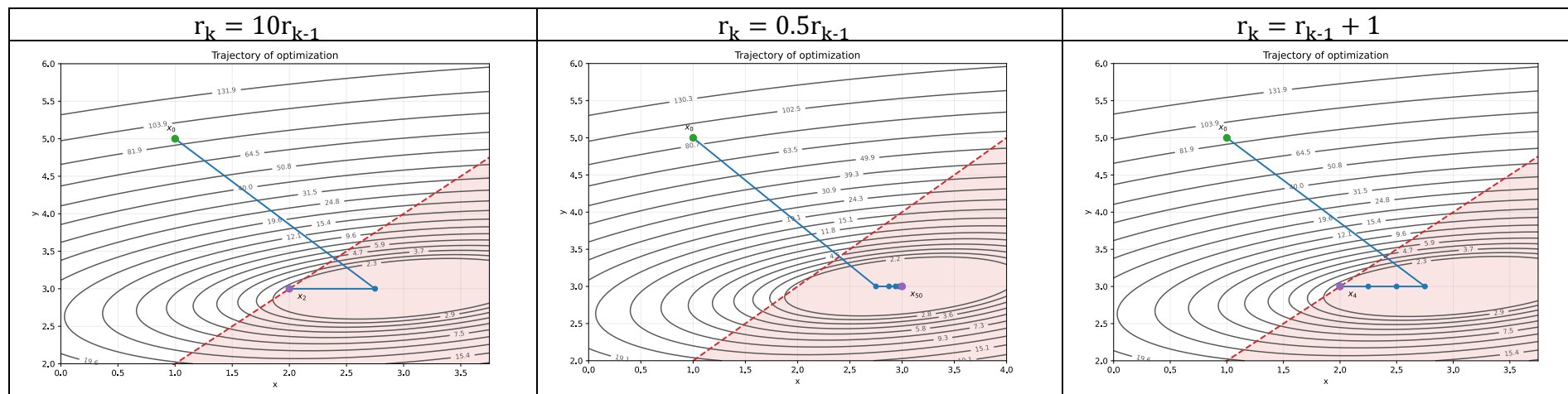
(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
(1,5)	1e-5	1	$r_k = 10r_{k-1}$	2	822	1.999997933	2.999998	1.999999
		5		1	631	2.035715241	3.035713	2.017845
		10		1	7187	2.213988805	3.213989	2.641077



Увеличение r_0 ухудшает результат и увеличивает количество вычислений, поэтому самое оптимальное $r_0 = 1$.

3.1.3 Зависимость от стратегии изменения γ

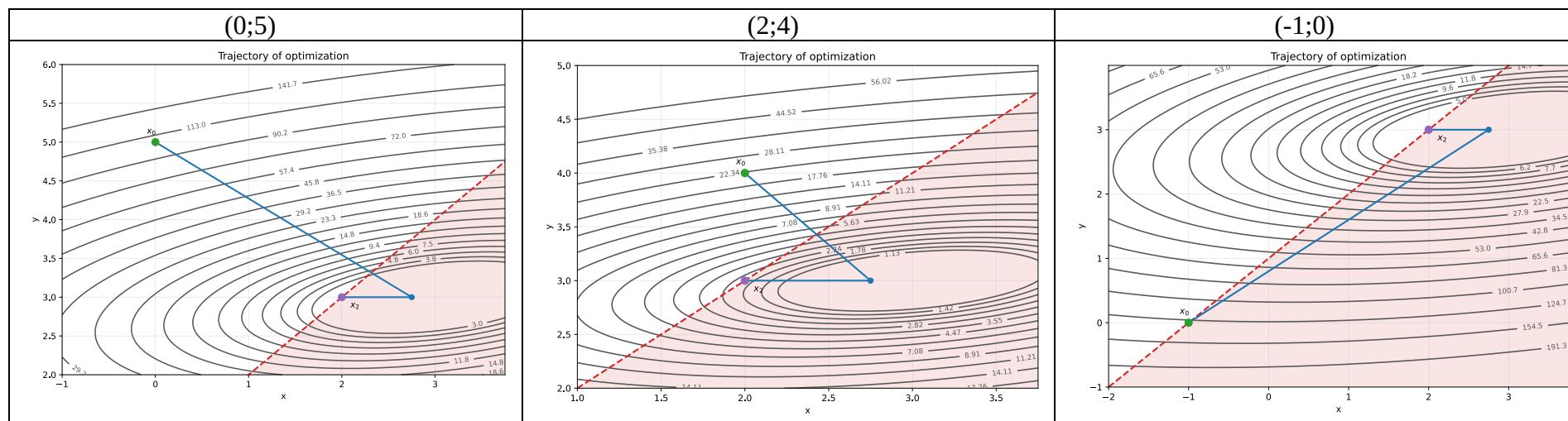
(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
(1,5)	1e-5	1	$r_k = 10r_{k-1}$	2	822	1.999997933	3.00E+00	2
			$r_k = 0.5r_{k-1}$	50	6828	2.999999738	2.999999	4.09E-12
			$r_k = r_{k-1} + 1$	4	1195	1.999999912	3	2



При уменьшении коэффициента штрафа ($r_k = 0.5r_{k-1}$) функция штрафа минимально “штрафует” решение и из-за этого на решение глобально она не влияет. А при сильном росте коэффициента штрафа ($r_k = 10r_{k-1}$ или $r_k = r_{k-1} + 1$) наглядно показывается влияние штрафной функции.

3.1.4 Зависимость от начальной точки

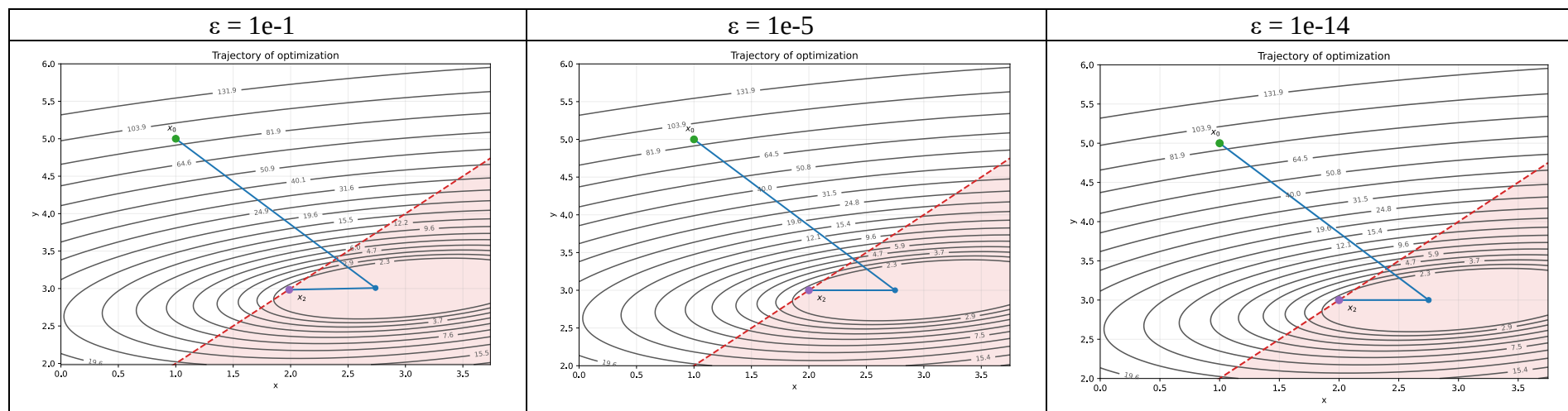
(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x,y)
(0;5)	1e-5	1	$r_k = 10r_{k-1}$	2	808	1.999993675	2.999992	1.999995
(2;4)				2	839	1.999998915	2.999999	1.999999
(-1;0)				2	909	1.99999817	2.999998	1.999999



Чем дальше точка, тем больше итераций нужно для поиска решения.

3.1.5 Зависимость от точности ε

(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x,y)
(1;5)	1e-1	1	$r_k = 10r_{k-1}$	2	224	1.987725199	2.983946	1.988521
	1e-5			2	822	1.999997933	2.999998	1.999999
	1e-14			2	2811965	1.999999998	3	2



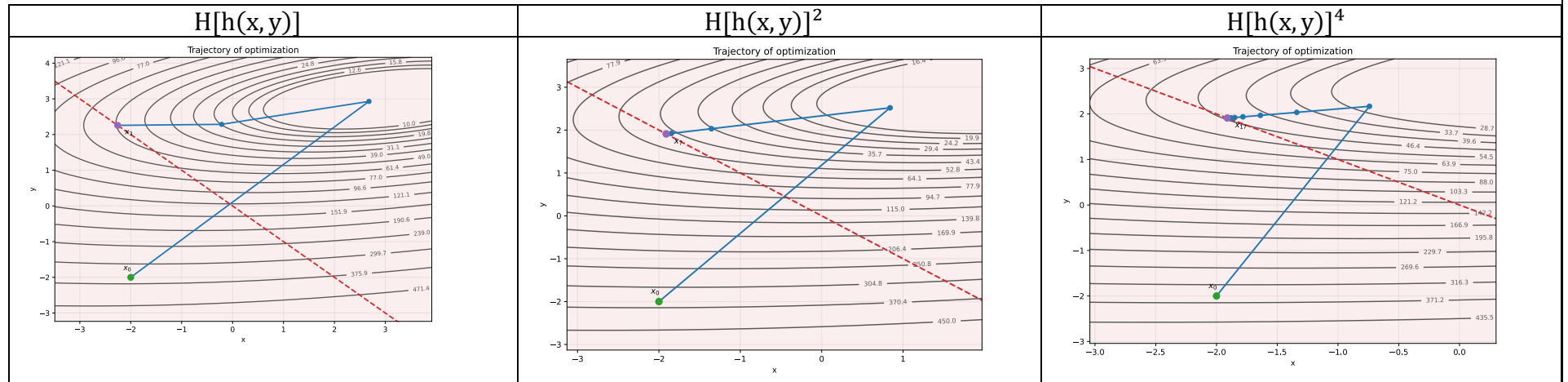
Чем меньше ε , тем больше вычислений, итоговая точка изменится слабо.

3.2 Метод штрафных функций задача (б)

3.2.1 Зависимость от выбора штрафных функции

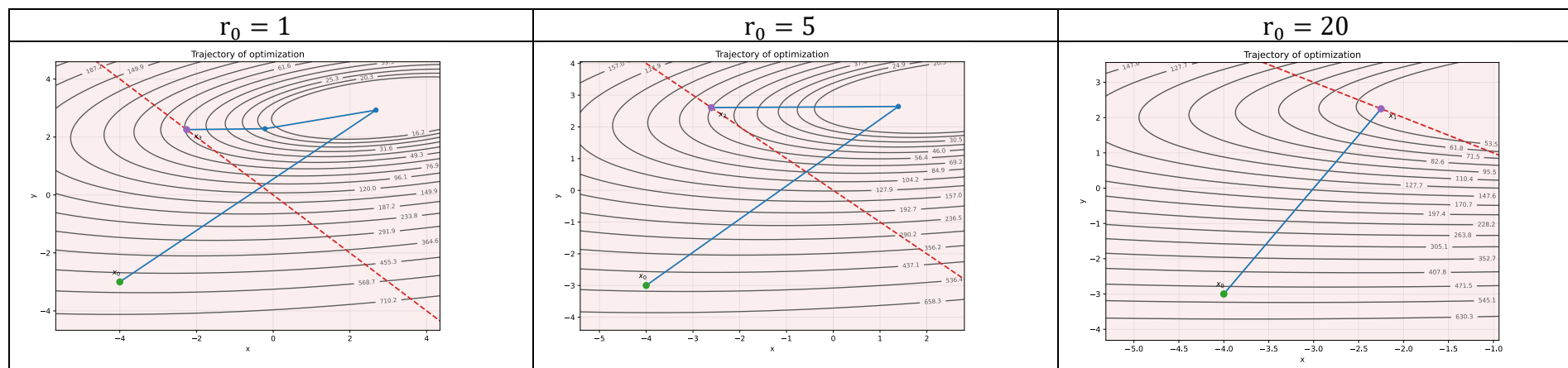
$$H[h(x, y)] = |h(x, y)|$$

Штрафные ф-и	(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
$H[h(x, y)]$	(-2;-2)	1e-5	1	$r_k = 10r_{k-1}$	3	921359	-2.257105707	2.257104	48.4827
$H[h(x, y)]^2$					7	17801	-1.910877823	1.910884	45.81816
$H[h(x, y)]^4$					17	41851	-1.911546427	1.911552	45.81823



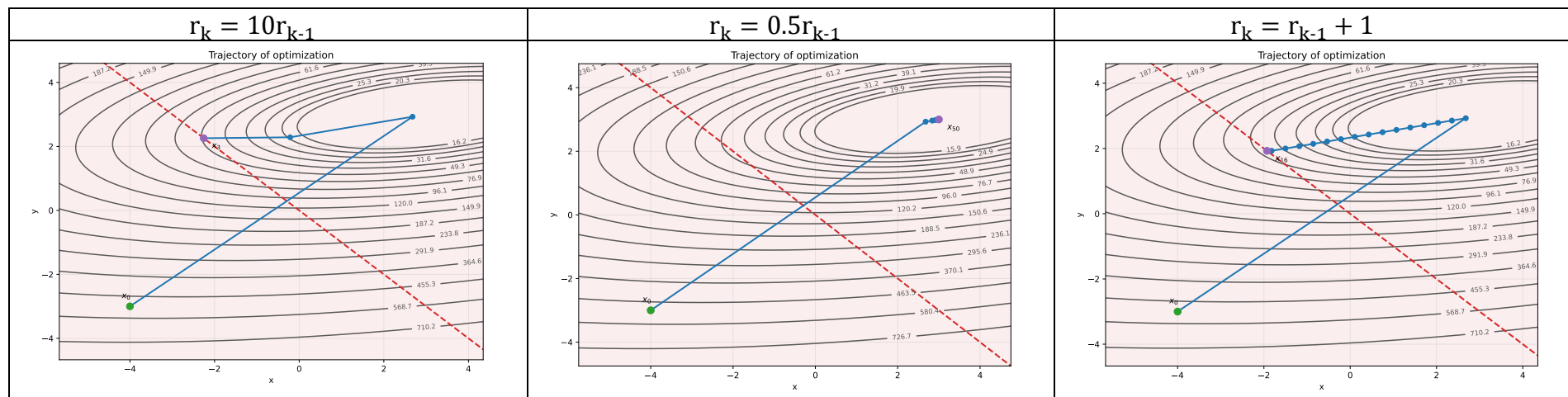
3.2.2 Зависимость от r_0

(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
(-4, -3)	1e-5	1	$r_k = 10r_{k-1}$	3	919776	-2.256571151	2.25657	48.47452
		5		2	900719	-2.607492933	2.607492	56.54901
		20		1	288	-2.249998764	2.249999	48.37498



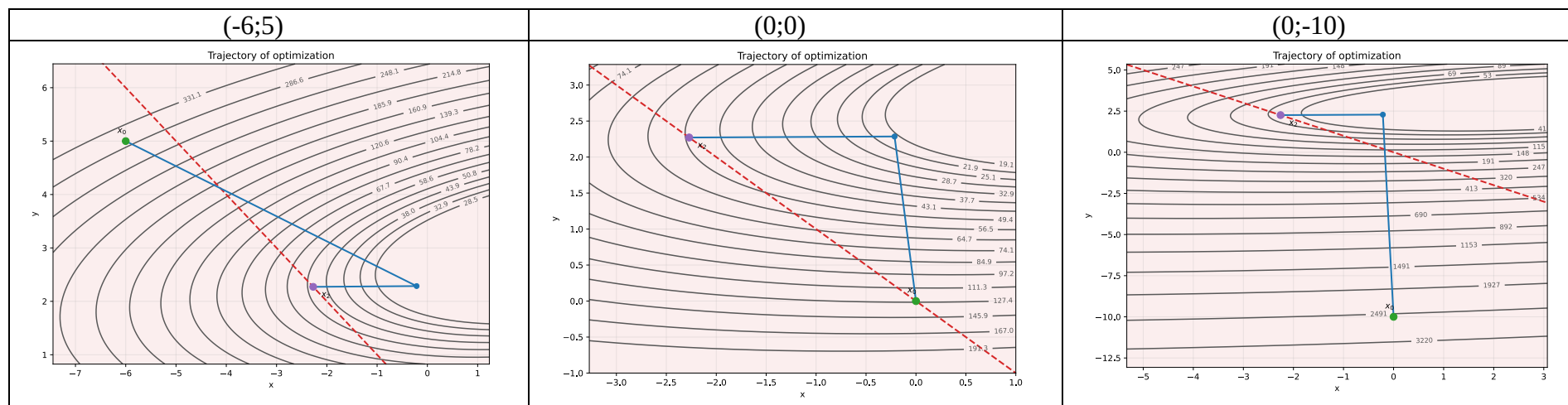
3.2.3 Зависимость от стратегии изменения γ

(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
(-4, -3)	1e-5	1	$r_k = 10r_{k-1}$	3	919776	-2.256571151	2.25657	48.47452
			$r_k = 0.5r_{k-1}$	50	9187	3.000001931	2.999999	4.59E-11
			$r_k = r_{k-1} + 1$	16	8944	-1.928569561	1.92857	45.82653



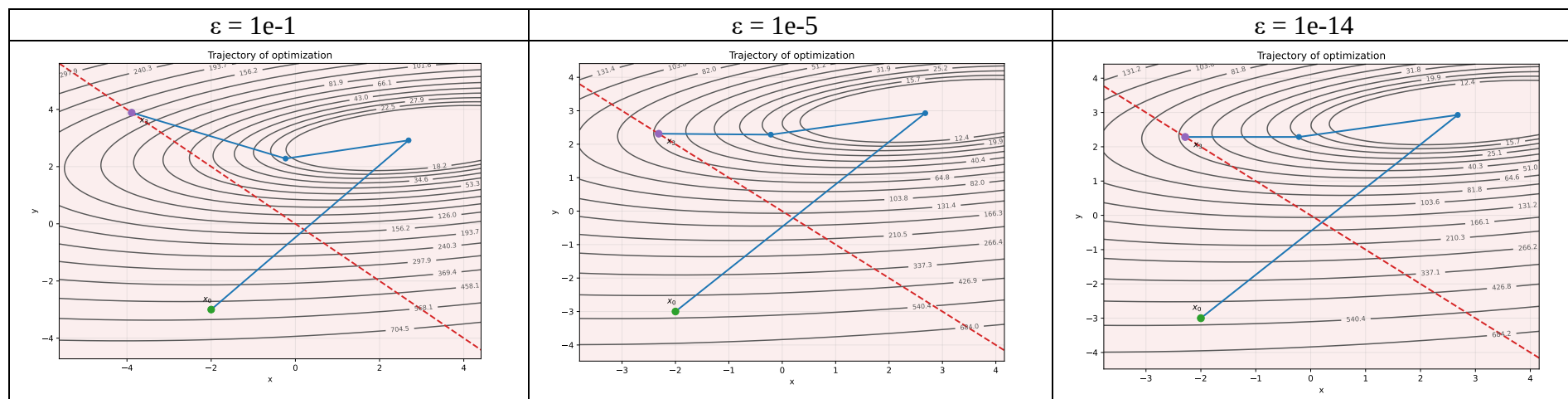
3.2.4 Зависимость от начальной точки

(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
$(-6; 5)$	$1e-5$	10	$r_k = 10r_{k-1}$	2	960622	-2.271410056	2.271409	48.70624
$(0; 0)$				2	960632	-2.271409006	2.271408	48.70622
$(0; -10)$				2	920635	-2.257106917	2.257105	48.48272



3.2.5 Зависимость от точности ε

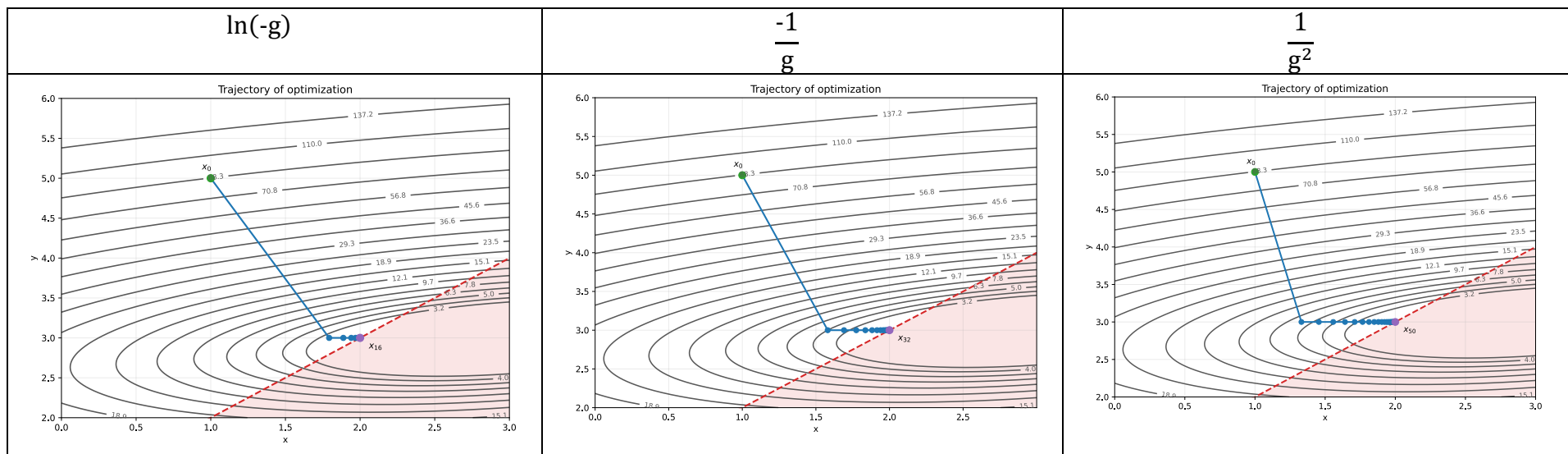
(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
(-2;-3)	1e-1	1	$r_k = 10r_{k-1}$	3	22505	-3.88812389	3.888067	131.9795
	1e-5			3	921353	-2.314323313	2.314325	49.43087
	1e-14			3	5194999	-2.285714266	2.285714	48.93878



3.3 Метод барьерных функций (а)

3.3.1 Зависимость от выбора барьерных функций

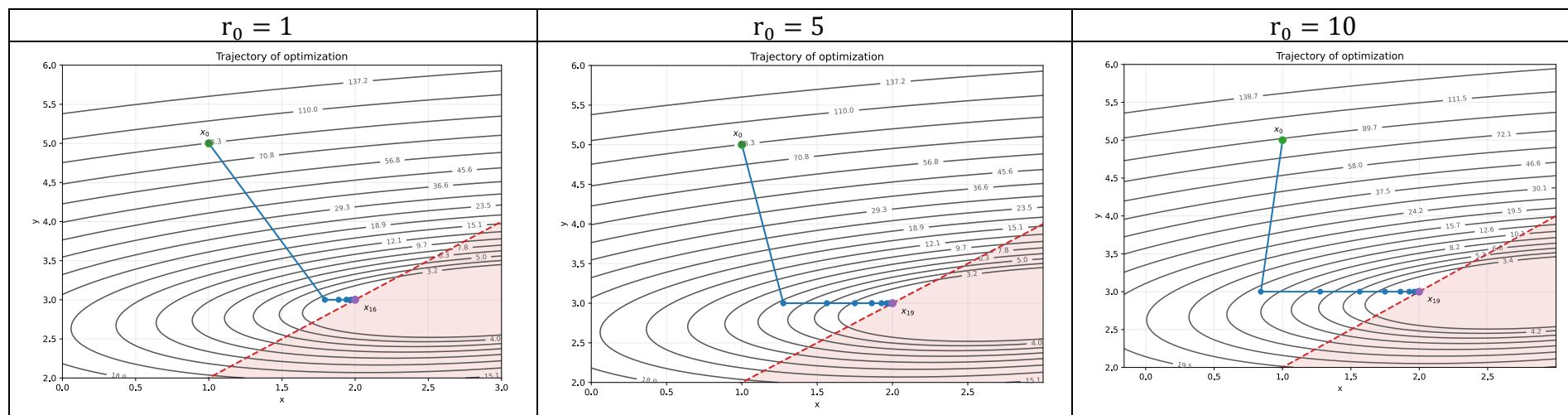
Барьерные ф-и	(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
$-\ln(-g)$	(1;5)	$1e-5$	1	$r_k = \frac{r_{k-1}}{2}$	16	4843	2.000010195	3.000019	2.000035
$-\frac{1}{g}$					32	7437	1.999989784	2.999999	2.000038
$\frac{1}{g^2}$					50	10485	2.00003638	3.000048	2.000045



Логарифм показывает стабильное решение системы. Остальные функции показывают чувствительный и тяжёлый расчёт(требуют больше итераций)

3.3.2 Зависимость от r_0

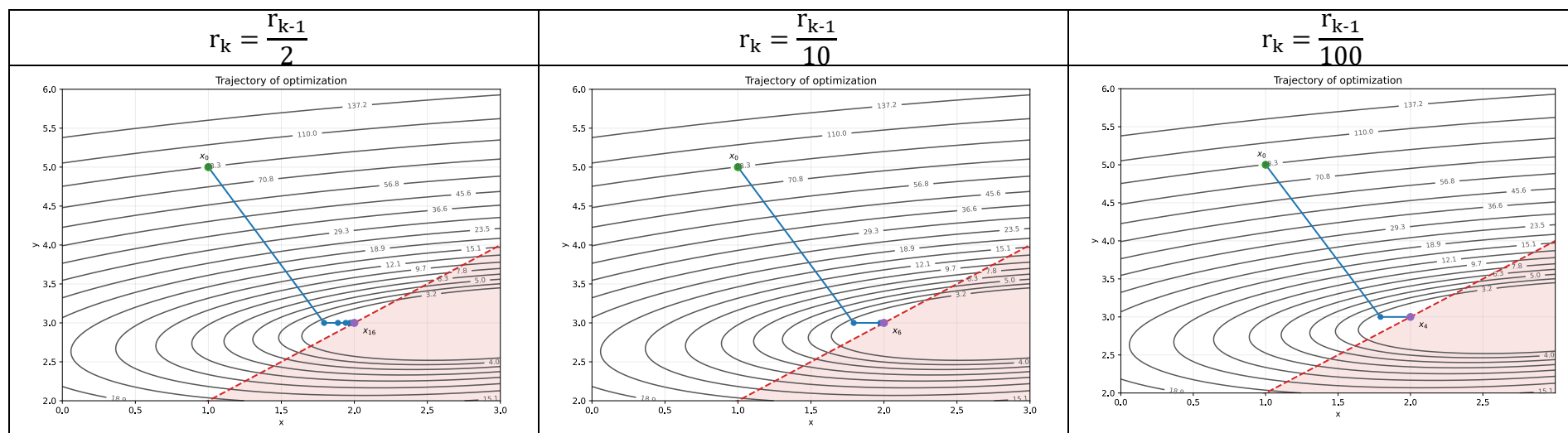
(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
(1,5)	1e-5	1	$r_k = \frac{r_{k-1}}{2}$	16	4843	2.000010195	3.000019	2.000035
		5		19	4309	1.999996514	3	2.000015
		10		19	4315	1.999988975	2.999998	2.000036



При увеличении r_0 возрастает количество требуемых итераций, но не критично.

3.3.3 Зависимость от стратегии изменения γ

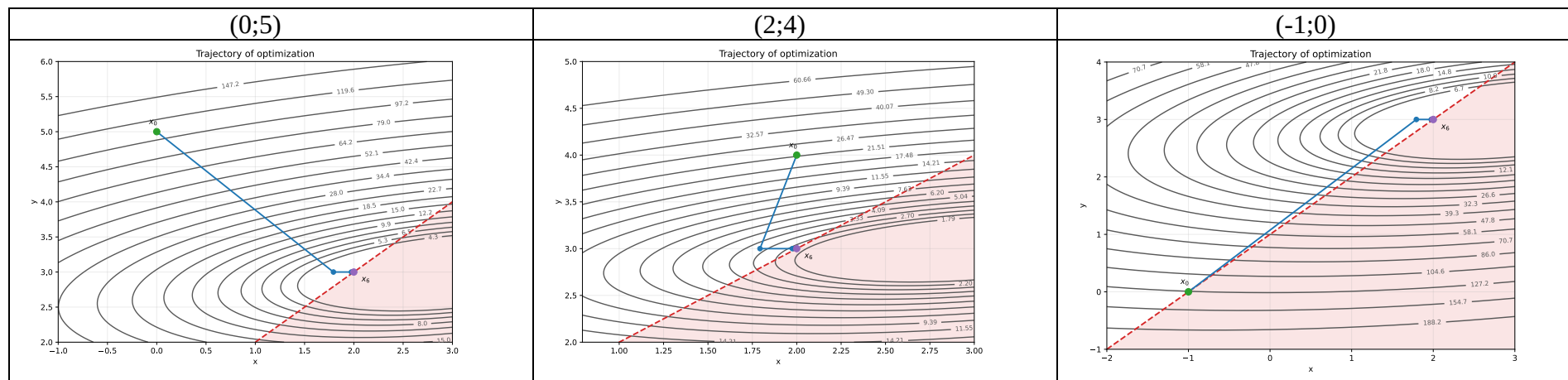
(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x, y)
(1,5)	1e-5	1	$r_k = \frac{r_{k-1}}{2}$	16	4843	2.000010195	3.000019	2.000035
			$r_k = \frac{r_{k-1}}{10}$	6	3056	2.000020521	3.000025	2.000017
			$r_k = \frac{r_{k-1}}{100}$	4	2875	1.999990255	2.99999	2.000001



Чем “быстрее” убывает r_k , тем быстрее сходится решение.

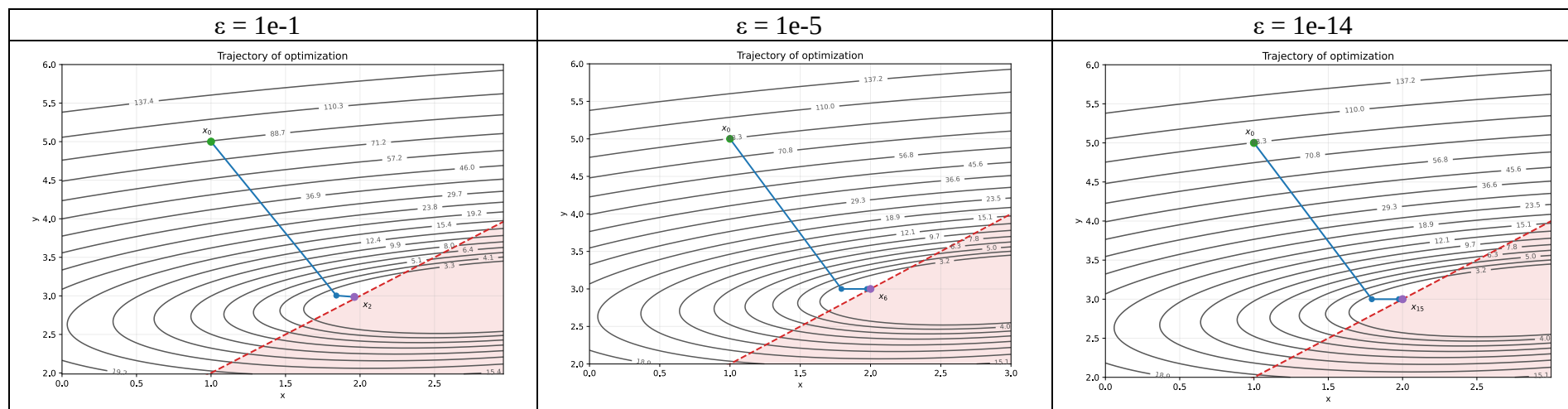
3.3.4 Зависимость от начального приближения

(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x,y)
(0;5)	1e-5	1	$r_k = \frac{r_{k-1}}{10}$	6	2984	2.000000612	3.0000004	2.000013
(2;4)				6	2653	1.999989751	2.999991	2.000007
(-1;0)				6	2256	2.000019434	3.000024	2.000016



3.3.5 Зависимость от точности ε

(x_0, y_0)	ε	r_0	r_k	Итераций	Вычисления функции	min x	min y	Функция в min(x,y)
(1;5)	1e-1	1	$r_k = \frac{r_{k-1}}{10}$	2	594	1.96208791	2.986462	2.101249
	1e-5			6	3056	2.000020521	3.000025	2.000017
	1e-14			15	29145663	1.999999641	3	2



4 Вывод

Оба метода позволяют решать задачи нелинейного программирования с ограничениями методом 0-го порядка, однако их эффективность сильно зависит от выбора вспомогательной функции и параметров алгоритма. Для штрафного метода ключевым является правильный рост коэффициента штрафа, а для барьерного метода - корректное уменьшение барьерного параметра и допустимое начальное приближение. Наиболее заметно на стоимость вычислений влияет требуемая точность ε , а на качество и стабильность сходимости - вид штрафной или барьерной функции.

5 Программа

5.1 Main.cpp

```
#include "PenaltyBarrierMethod.h"

#include <cstdlib>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>

namespace fs = std::filesystem;

static fs::path gProjectRoot;

static fs::path FindProjectRoot()
{
    fs::path cur = fs::current_path();
    while (!cur.empty())
    {
        if (fs::exists(cur / "in.txt") && fs::exists(cur / "M0_3.vcxproj"))
            return cur;
        fs::path parent = cur.parent_path();
        if (parent == cur)
            break;
        cur = parent;
    }
    return fs::current_path();
}

static const char* MethodName(int methodType)
```

```

{
    return (methodType == 1) ? "penalty" : "barrier";
}

static const char* ProblemName(int methodType, int problemType)
{
    if (methodType == 1)
        return (problemType == 1) ? "task_a" : "task_b";
    return "task_a";
}

static const char* ModifierName(int methodType, int variant)
{
    if (methodType == 1)
    {
        switch (variant)
        {
            case 1: return "max(0,g)";
            case 2: return "max(0,g)^2";
            case 3: return "max(0,g)^4";
            default: return "max(0,g)^2";
        }
    }

    switch (variant)
    {
        case 1: return "-ln(-g)";
        case 2: return "-1/g";
        case 3: return "1/g^2";
        default: return "-ln(-g)";
    }
}

static const char* StrategyName(int methodType, int variant)
{
    if (methodType == 1)
    {
        switch (variant)
        {
            case 1: return "r_{k+1}=beta*r_k";
            case 2: return "r_{k+1}=r_k+beta";
            case 3: return "r_{k+1}=r_k+1";
            default: return "r_{k+1}=beta*r_k";
        }
    }
}

```

```

    }
}

switch (variant)
{
case 1: return "r_{k+1}=r_k/beta";
case 2: return "r_{k+1}=max(r_k-beta,eps)";
case 3: return "r_{k+1}=r_k/2";
default: return "r_{k+1}=r_k/beta";
}
}

static void WriteSummaryHeaderIfNeeded()
{
    const fs::path summaryFile = gProjectRoot / "out1.txt";
    bool rewrite = true;
    if (fs::exists(summaryFile))
    {
        std::ifstream in(summaryFile);
        std::string firstLine;
        std::getline(in, firstLine);
        rewrite = firstLine.find("strategy_r") == std::string::npos || firstLine.find("func_evals") == std::string::npos;
    }

    if (rewrite)
    {
        std::ofstream out(summaryFile, std::ios::trunc);
        out
"method\tproblem\tmodifier\tstrategy_r\tstart_x\tstart_y\teps\ttr0\tbeta\titers\tinner_iters\tfunc_evals\tresult_x\tresult_y\tobjective
\tviolation_or_gap\tcoefficient" << std::endl;
    }
}

static void AppendSummary(int methodType, int problemType, int modifierVariant, int strategyVariant, const double* startPoint,
    double eps, double r0, double beta, int outerIters, int innerIters, long long funcEvals,
    double resultX, double resultY, double objective, double measure, double coefficient)
{
    WriteSummaryHeaderIfNeeded();
    std::ofstream out(gProjectRoot / "out1.txt", std::ios::app);
    out << MethodName(methodType) << '\t'
        << ProblemName(methodType, problemType) << '\t'
        << ModifierName(methodType, modifierVariant) << '\t'
        << StrategyName(methodType, strategyVariant) << '\t'

```

```

        << std::setprecision(16) << startPoint[0] << '\t'
        << startPoint[1] << '\t'
        << eps << '\t'
        << r0 << '\t'
        << beta << '\t'
        << outerIters << '\t'
        << innerIters << '\t'
        << funcEvals << '\t'
        << resultX << '\t'
        << resultY << '\t'
        << objective << '\t'
        << measure << '\t'
        << coefficient << std::endl;
    }

static void RunPlotter()
{
    fs::path script = gProjectRoot.parent_path() / "Graphiks_M03" / "Graphiks_M03.py";
    if (!fs::exists(script))
        return;

    std::string command = "py \"" + script.string() + "\"";
    if (std::system(command.c_str()) != 0)
    {
        command = "python \"" + script.string() + "\"";
        std::system(command.c_str());
    }
}

int main()
{
    gProjectRoot = FindProjectRoot();
    fs::current_path(gProjectRoot);

    PenaltyBarrierMethod solver;
    int methodType;
    int problemType = 1;
    int modifierVariant = 2;
    int strategyVariant = 1;
    double startPoint[2];
    double eps;
    double r0 = 1.0;
    double beta = 10.0;

```

```

std::ifstream input(gProjectRoot / "in.txt");

if (!input)
{
    std::cerr << "Cannot open in.txt" << std::endl;
    return 1;
}

if (!(input >> startPoint[0] >> startPoint[1] >> eps))
{
    std::cerr << "Invalid format in.txt. Expected: x0 y0 eps [r0 beta]" << std::endl;
    return 1;
}

if (!(input >> r0 >> beta))
{
    r0 = 1.0;
    beta = 10.0;
}

std::cout << "Method (1 - penalty, 2 - barrier): ";
std::cin >> methodType;

if (methodType == 1)
{
    std::cout << "Penalty problem (1 - task a:  $y - x \geq 1$ , 2 - task b:  $x = -y$ ): ";
    std::cin >> problemType;
    std::cout << "Penalty function (1 -  $\max(0, g)$ , 2 -  $\max(0, g)^2$ , 3 -  $\max(0, g)^4$ ): ";
    std::cin >> modifierVariant;
    std::cout << "Penalty r strategy (1 -  $\beta \cdot r_k$ , 2 -  $r_k + \beta$ , 3 -  $r_{k+1}$ ): ";
    std::cin >> strategyVariant;

    solver.InitPenalty(2, problemType);
    solver.SetModifierVariant(modifierVariant);
    solver.SetStrategyVariant(strategyVariant);
    solver.SetStartPoint(startPoint);
    solver.SetTolerance(eps);
    solver.SetParameters(r0, beta);
}
else
{
    problemType = 1;
    std::cout << "Barrier function (1 -  $-\ln(-g)$ , 2 -  $-1/g$ , 3 -  $1/g^2$ ): ";
}

```

```

        std::cin >> modifierVariant;
        std::cout << "Barrier r strategy (1 - r_k/beta, 2 - max(r_k-beta,eps), 3 - r_k/2): ";
        std::cin >> strategyVariant;

        solver.InitBarrier(2);
        solver.SetModifierVariant(modifierVariant);
        solver.SetStrategyVariant(strategyVariant);
        solver.SetStartPoint(startPoint);
        solver.SetTolerance(eps);
        solver.SetParameters(r0, beta);
    }

    double* res = solver.Solver();

    std::cout << "\nResult: ";
    for (int i = 0; i < 2; ++i)
        std::cout << std::setprecision(16) << res[i] << " ";
    std::cout << "\nOuter iterations: " << solver.GetLastOuterIterationCount();
    std::cout << "\nTotal inner iterations: " << solver.GetTotalInnerIterationCount();
    std::cout << "\nObjective value: " << std::setprecision(16) << solver.GetLastObjectiveValue();
    std::cout << "\n" << ((methodType == 1) ? "Constraint violation: " : "Boundary gap: ")
        << std::setprecision(16) << solver.GetLastMeasure();
    std::cout << "\nLast coefficient: " << std::setprecision(16) << solver.GetLastCoefficient();
    std::cout << "\nFunction evaluations: " << solver.GetTotalFunctionEvaluationCount() << std::endl;

    AppendSummary(methodType, problemType, modifierVariant, strategyVariant, startPoint, eps, r0, beta,
        solver.GetLastOuterIterationCount(), solver.GetTotalInnerIterationCount(), solver.GetTotalFunctionEvaluationCount(),
        res[0], res[1], solver.GetLastObjectiveValue(), solver.GetLastMeasure(), solver.GetLastCoefficient());

    RunPlotter();
    return 0;
}

```

5.2 PenaltyBarrierMethod.cpp

```

#include "PenaltyBarrierMethod.h"

#include <fstream>
#include <iomanip>
#include <limits>

PenaltyBarrierMethod::PenaltyBarrierMethod()

```



```

{
    mode = Mode::Penalty;
    n = 2;
    x0 = nullptr;
    eps = 1e-5;
    r0 = 1.0;
    beta = 10.0;
    modifierVariant = 2;
    strategyVariant = 1;
    maxOuterIter = 50;
    lastOuterIterCount = 0;
    totalInnerIterCount = 0;
    totalFunctionEvalCount = 0;
    lastObjectiveValue = 0.0;
    lastMeasure = 0.0;
    lastCoefficient = 0.0;
}

PenaltyBarrierMethod::~PenaltyBarrierMethod()
{
    delete[] x0;
}

void PenaltyBarrierMethod::InitPenalty(int n1, int constraintType)
{
    mode = Mode::Penalty;
    n = n1;
    delete[] x0;
    x0 = new double[n];
    for (int i = 0; i < n; ++i)
        x0[i] = -1.0;

    gauss.Init(n);
    gauss.SetFunctionType(4);
    gauss.SetConstraintType(constraintType);
    gauss.SetModifierType(1);
    gauss.SetModifierVariant(modifierVariant);

    baseFunc.SetFunctionType(4);
    baseFunc.SetConstraintType(constraintType);
}

void PenaltyBarrierMethod::InitBarrier(int n1)

```

```

{
    mode = Mode::Barrier;
    n = n1;
    delete[] x0;
    x0 = new double[n];
    for (int i = 0; i < n; ++i)
        x0[i] = -1.0;

    gauss.Init(n);
    gauss.SetFunctionType(4);
    gauss.SetConstraintType(1);
    gauss.SetModifierType(2);
    gauss.SetModifierVariant(modifierVariant);

    baseFunc.SetFunctionType(4);
    baseFunc.SetConstraintType(1);
}

void PenaltyBarrierMethod::SetStartPoint(const double* startPoint)
{
    for (int i = 0; i < n; ++i)
        x0[i] = startPoint[i];
}

void PenaltyBarrierMethod::SetTolerance(double epsValue)
{
    eps = epsValue;
    gauss.SetTolerances(epsValue);
}

void PenaltyBarrierMethod::SetParameters(double startPenalty, double growthFactor)
{
    r0 = startPenalty;
    beta = growthFactor;
}

void PenaltyBarrierMethod::SetModifierVariant(int type)
{
    modifierVariant = type;
    gauss.SetModifierVariant(type);
}

void PenaltyBarrierMethod::SetStrategyVariant(int type)

```

```

{
    strategyVariant = type;
}

int PenaltyBarrierMethod::GetLastOuterIterationCount() const
{
    return lastOuterIterCount;
}

int PenaltyBarrierMethod::GetTotalInnerIterationCount() const
{
    return totalInnerIterCount;
}

long long PenaltyBarrierMethod::GetTotalFunctionEvaluationCount() const
{
    return totalFunctionEvalCount;
}

double PenaltyBarrierMethod::GetLastObjectiveValue() const
{
    return lastObjectiveValue;
}

double PenaltyBarrierMethod::GetLastMeasure() const
{
    return lastMeasure;
}

double PenaltyBarrierMethod::GetLastCoefficient() const
{
    return lastCoefficient;
}

void PenaltyBarrierMethod::UpdateCoefficient()
{
    if (mode == Mode::Penalty)
    {
        switch (strategyVariant)
        {
            {
                case 1: lastCoefficient *= beta; break;
                case 2: lastCoefficient += beta; break;
                case 3: lastCoefficient += 1.0; break;
            }
        }
    }
}

```

```

        default: lastCoefficient *= beta; break;
    }
    return;
}

switch (strategyVariant)
{
case 1:
    lastCoefficient /= beta;
    break;
case 2:
    lastCoefficient -= beta;
    if (lastCoefficient <= 1e-12)
        lastCoefficient = 1e-12;
    break;
case 3:
    lastCoefficient /= 2.0;
    break;
default:
    lastCoefficient /= beta;
    break;
}
}

double* PenaltyBarrierMethod::Solver()
{
    const bool penaltyMode = (mode == Mode::Penalty);
    std::ofstream file(penaltyMode ? "out_penalty.txt" : "out_barrier.txt");
    std::ofstream traj(penaltyMode ? "out_penalty_traj.txt" : "out_barrier_traj.txt");

    if (penaltyMode)
        file << "outer\tr\tx\ty\tf\tviolation\tinner_iters" << std::endl;
    else
        file << "outer\tr\tx\ty\tf\tboundary_gap\tinner_iters" << std::endl;

    traj << 0 << ' ' << x0[0] << ' ' << x0[1] << std::endl;

    gauss.SetStartPoint(x0);
    baseFunc.ResetEvaluationCount();
    totalInnerIterCount = 0;
    totalFunctionEvalCount = 0;
    lastOuterIterCount = 0;
    lastCoefficient = r0;
}

```

```

for (int outerIter = 1; outerIter <= maxOuterIter; ++outerIter)
{
    gauss.SetCoefficient(lastCoefficient);
    double* current = gauss.Solver();
    totalFunctionEvalCount += gauss.GetFunctionEvaluationCount();

    for (int i = 0; i < n; ++i)
        x0[i] = current[i];

    lastObjectiveValue = baseFunc.Objective(current);
    ++totalFunctionEvalCount;
    lastMeasure = penaltyMode ? baseFunc.ConstraintViolation(current) : -baseFunc.ConstraintValue(current);
    totalInnerIterCount += gauss.GetLastIterationCount();
    lastOuterIterCount = outerIter;

    file << outerIter << "\t"
        << std::setprecision(16) << lastCoefficient << "\t"
        << current[0] << "\t"
        << current[1] << "\t"
        << lastObjectiveValue << "\t"
        << lastMeasure << "\t"
        << gauss.GetLastIterationCount() << std::endl;
    traj << outerIter << ' ' << std::setprecision(16) << current[0] << ' ' << current[1] << std::endl;

    if (lastMeasure <= eps)
        return current;

    gauss.SetStartPoint(current);
    UpdateCoefficient();
}

double* current = gauss.Solver();
totalFunctionEvalCount += gauss.GetFunctionEvaluationCount();
return current;
}

```

5.3 PenaltyBarrierMethod.h

```

#pragma once

#include "Gauss.h"

```

```

class PenaltyBarrierMethod
{
public:
    enum class Mode
    {
        Penalty,
        Barrier
    };

private:
    Mode mode;
    int n;
    double* x0;
    double eps;
    double r0;
    double beta;
    int modifierVariant;
    int strategyVariant;
    int maxOuterIter;
    int lastOuterIterCount;
    int totalInnerIterCount;
    long long totalFunctionEvalCount;
    double lastObjectiveValue;
    double lastMeasure;
    double lastCoefficient;
    Gauss gauss;
    Func baseFunc;

    void UpdateCoefficient();

public:
    PenaltyBarrierMethod();
    ~PenaltyBarrierMethod();

    void InitPenalty(int n1, int constraintType);
    void InitBarrier(int n1);
    void SetStartPoint(const double* startPoint);
    void SetTolerance(double epsValue);
    void SetParameters(double startPenalty, double growthFactor);
    void SetModifierVariant(int type);
    void SetStrategyVariant(int type);

```

```

    int GetLastOuterIterationCount() const;
    int GetTotalInnerIterationCount() const;
    long long GetTotalFunctionEvaluationCount() const;
    double GetLastObjectiveValue() const;
    double GetLastMeasure() const;
    double GetLastCoefficient() const;

    double* Solver();
};

```

5.4 Gauss.cpp

```

#include "Gauss.h"
#include <algorithm>
#include <cmath>
#include <fstream>
#include <iomanip>

Gauss::Gauss()
{
    n = 2;
    xk = nullptr;
    xk_prev = nullptr;
    x0 = nullptr;
    eps_f = 1e-7;
    eps_x = 1e-7;
    maxIter = 10000;
    lastIterCount = 0;
}

Gauss::~~Gauss()
{
    delete[] xk;
    delete[] xk_prev;
    delete[] x0;
}

void Gauss::Init(int n1)
{
    n = n1;
    x0 = new double[n];
    xk = new double[n];
}

```

```

    xk_prev = new double[n];

    for (int i = 0; i < n; i++)
    {
        x0[i] = -1.0;
        xk[i] = x0[i];
        xk_prev[i] = x0[i];
    }
}

void Gauss::SetFunctionType(int type)
{
    f.SetFunctionType(type);
}

void Gauss::SetConstraintType(int type)
{
    f.SetConstraintType(type);
}

void Gauss::SetModifierType(int type)
{
    f.SetModifierType(type);
}

void Gauss::SetModifierVariant(int type)
{
    f.SetModifierVariant(type);
}

void Gauss::SetCoefficient(double value)
{
    f.SetCoefficient(value);
}

void Gauss::SetStartPoint(const double* startPoint)
{
    for (int i = 0; i < n; i++)
    {
        x0[i] = startPoint[i];
        xk[i] = startPoint[i];
        xk_prev[i] = startPoint[i];
    }
}

```



```

}

void Gauss::SetTolerances(double epsValue)
{
    eps_f = epsValue;
    eps_x = epsValue;
}

int Gauss::GetLastIterationCount() const
{
    return lastIterCount;
}

long long Gauss::GetFunctionEvaluationCount() const
{
    return f.GetEvaluationCount();
}

double Gauss::GetCurrentFunctionValue() const
{
    return f.func(xk);
}

void Gauss::FindInterval(int coord, double& a, double& b)
{
    const double initialStep = 1.0;
    const double minStep = 1e-12;
    const double expandFactor = 2.0;

    double h = initialStep;
    const double f0 = f.funcInDirection(xk, coord, 0.0);
    double fPlus = f.funcInDirection(xk, coord, h);
    double fMinus = f.funcInDirection(xk, coord, -h);

    while (fPlus >= f0 && fMinus >= f0 && h > minStep)
    {
        h *= 0.5;
        fPlus = f.funcInDirection(xk, coord, h);
        fMinus = f.funcInDirection(xk, coord, -h);
    }

    if (h <= minStep)
    {

```

```

        a = -initialStep;
        b = initialStep;
        return;
    }

    const double direction = (fPlus < fMinus) ? 1.0 : -1.0;
    double lambdaPrev = 0.0;
    double lambdaCurr = direction * h;
    double fCurr = f.funcInDirection(xk, coord, lambdaCurr);

    for (int iter = 0; iter < 100; iter++)
    {
        const double lambdaNext = lambdaCurr + direction * h * expandFactor;
        const double fNext = f.funcInDirection(xk, coord, lambdaNext);

        if (fNext >= fCurr)
        {
            a = std::min(lambdaPrev, lambdaNext);
            b = std::max(lambdaPrev, lambdaNext);
            return;
        }

        lambdaPrev = lambdaCurr;
        lambdaCurr = lambdaNext;
        fCurr = fNext;
        h *= expandFactor;
    }

    a = std::min(lambdaPrev, lambdaCurr);
    b = std::max(lambdaPrev, lambdaCurr);
}

double Gauss::GoldenSection(int coord, double left, double right) const
{
    const double phi = (1.0 + std::sqrt(5.0)) / 2.0;
    double x1 = right - (right - left) / phi;
    double x2 = left + (right - left) / phi;
    double f1 = f.funcInDirection(xk, coord, x1);
    double f2 = f.funcInDirection(xk, coord, x2);

    while (std::fabs(right - left) > eps_x)
    {
        if (f1 <= f2)

```

```

    {
        right = x2;
        x2 = x1;
        f2 = f1;
        x1 = right - (right - left) / phi;
        f1 = f.funcInDirection(xk, coord, x1);
    }
    else
    {
        left = x1;
        x1 = x2;
        f1 = f2;
        x2 = left + (right - left) / phi;
        f2 = f.funcInDirection(xk, coord, x2);
    }
}

return 0.5 * (left + right);
}

double* Gauss::Solver()
{
    std::ofstream file1("gauss_trace.txt");
    std::ofstream file2("out2.txt");
    file1 << "i\tx\ty\tf\tlambda_x\tlambda_y\t|dx|\t|dy|\t|df|" << std::endl;
    file2 << 0 << ' ' << xk[0] << ' ' << xk[1] << std::endl;

    f.ResetEvaluationCount();
    double fPrev = f.func(xk);
    int iter = 0;
    lastIterCount = 0;

    while (iter < maxIter)
    {
        for (int i = 0; i < n; i++)
            xk_prev[i] = xk[i];

        const double fPrevIter = f.func(xk);
        double lambdas[2] = { 0.0, 0.0 };

        for (int coord = 0; coord < n; coord++)
        {
            double a = 0.0;

```

```

        double b = 0.0;
        FindInterval(coord, a, b);
        const double lambda = GoldenSection(coord, a, b);
        xk[coord] += lambda;
        lambdas[coord] = lambda;
    }

    const double fCur = f.func(xk);
    ++iter;

    file1 << iter << "\t"
        << std::setprecision(16) << xk[0] << "\t"
        << xk[1] << "\t"
        << fCur << "\t"
        << lambdas[0] << "\t"
        << lambdas[1] << "\t"
        << std::fabs(xk[0] - xk_prev[0]) << "\t"
        << std::fabs(xk[1] - xk_prev[1]) << "\t"
        << std::fabs(fCur - fPrevIter) << std::endl;

    file2 << iter << " "
        << std::setprecision(16) << xk[0] << " "
        << xk[1] << std::endl;

    const bool xSmall = std::fabs(xk[0] - xk_prev[0]) <= eps_x;
    const bool ySmall = std::fabs(xk[1] - xk_prev[1]) <= eps_x;

    if (std::fabs(fCur - fPrev) <= eps_f && xSmall && ySmall)
        break;

    fPrev = fCur;
}

lastIterCount = iter;
return xk;
}

```

5.5 Gauss.h

```

#pragma once
#include "Func.h"

```

```

class Gauss
{
private:
    int n;
    double* xk;
    double* xk_prev;
    double* x0;
    double eps_f;
    double eps_x;
    int maxIter;
    int lastIterCount;
    Func f;

    double GoldenSection(int coord, double left, double right) const;
    void FindInterval(int coord, double& a, double& b);
public:
    Gauss();
    ~Gauss();
    void Init(int n1);
    void SetFunctionType(int type);
    void SetConstraintType(int type);
    void SetModifierType(int type);
    void SetModifierVariant(int type);
    void SetCoefficient(double value);
    void SetStartPoint(const double* startPoint);
    void SetTolerances(double epsValue);
    int GetLastIterationCount() const;
    long long GetFunctionEvaluationCount() const;
    double GetCurrentFunctionValue() const;
    double* Solver();
};

```

5.6 Func.cpp

```

#include "Func.h"
#include <algorithm>
#include <cmath>
#include <limits>

void Func::SetFunctionType(int type)
{

```

```

    functionType = type;
}

void Func::SetConstraintType(int type)
{
    constraintType = type;
}

void Func::SetModifierType(int type)
{
    modifierType = type;
}

void Func::SetModifierVariant(int type)
{
    modifierVariant = type;
}

void Func::SetCoefficient(double value)
{
    coefficient = value;
}

void Func::ResetEvaluationCount()
{
    evaluationCount = 0;
}

long long Func::GetEvaluationCount() const
{
    return evaluationCount;
}

double Func::Objective(double* x) const
{
    ++evaluationCount;

    switch (functionType)
    {
    case 1:
        return 100.0 * (x[1] - x[0]) * (x[1] - x[0]) + (1.0 - x[0]) * (1.0 - x[0]);
    case 2:
        return 100.0 * (x[1] - x[0] * x[0]) * (x[1] - x[0] * x[0]) + (1.0 - x[0]) * (1.0 - x[0]);
    }
}

```

```

    case 3:
        return -2.0 / (1.0 + (x[0] - 1.0) * (x[0] - 1.0) / 4.0 + (x[1] - 2.0) * (x[1] - 2.0))
            - 1.0 / (1.0 + (x[0] - 3.0) * (x[0] - 3.0) / 9.0 + (x[1] - 1.0) * (x[1] - 1.0) / 9.0);
    case 4:
        return 2.0 * (x[0] - x[1]) * (x[0] - x[1]) + 14.0 * (x[1] - 3.0) * (x[1] - 3.0);
    default:
        return 100.0 * (x[1] - x[0] * x[0]) * (x[1] - x[0] * x[0]) + (1.0 - x[0]) * (1.0 - x[0]);
}

double Func::ConstraintValue(double* x) const
{
    switch (constraintType)
    {
    case 1:
        return x[0] - x[1] + 1.0;
    case 2:
        return x[0] + x[1];
    default:
        return 0.0;
    }
}

double Func::ConstraintViolation(double* x) const
{
    const double value = ConstraintValue(x);

    if (constraintType == 1)
        return std::max(0.0, value);

    if (constraintType == 2)
        return std::fabs(value);

    return 0.0;
}

double Func::func(double* x) const
{
    const double objective = Objective(x);

    if (modifierType == 0 || constraintType == 0)
        return objective;
}

```

```

    if (modifierType == 1)
    {
        const double violation = ConstraintViolation(x);
        switch (modifierVariant)
        {
            case 1:
                return objective + coefficient * violation;
            case 2:
                return objective + coefficient * violation * violation;
            case 3:
                return objective + coefficient * violation * violation * violation * violation;
            default:
                return objective + coefficient * violation * violation;
        }
    }

    if (modifierType == 2)
    {
        const double value = ConstraintValue(x);
        if (constraintType != 1 || value >= 0.0)
            return std::numeric_limits<double>::infinity();

        switch (modifierVariant)
        {
            case 1:
                return objective - coefficient * std::log(-value);
            case 2:
                return objective + coefficient * (-1.0 / value);
            case 3:
                return objective + coefficient * (1.0 / (value * value));
            default:
                return objective - coefficient * std::log(-value);
        }
    }

    return objective;
}

double Func::funcInDirection(double* xk, int coord, double lambda) const
{
    double point[2] = { xk[0], xk[1] };
    point[coord] += lambda;
    return func(point);
}

```



```
}
```

5.7 Func.h

```
#pragma once
```

```
class Func
```

```
{
```

```
    int functionType;
```

```
    int constraintType;
```

```
    int modifierType;
```

```
    int modifierVariant;
```

```
    double coefficient;
```

```
    mutable long long evaluationCount;
```

```
public:
```

```
    Func()
```

```
        : functionType(2), constraintType(0), modifierType(0), modifierVariant(2), coefficient(0.0), evaluationCount(0)
```

```
    {
```

```
    }
```

```
    void SetFunctionType(int type);
```

```
    void SetConstraintType(int type);
```

```
    void SetModifierType(int type);
```

```
    void SetModifierVariant(int type);
```

```
    void SetCoefficient(double value);
```

```
    void ResetEvaluationCount();
```

```
    long long GetEvaluationCount() const;
```

```
    double Objective(double* x) const;
```

```
    double ConstraintValue(double* x) const;
```

```
    double ConstraintViolation(double* x) const;
```

```
    double func(double* x) const;
```

```
    double funcInDirection(double* xk, int coord, double lambda) const;
```

```
};
```

5.8 VecOperations.cpp

```
#include "VecOperations.h"
```

```
double* VecOperations::MultVecScal(double* vec, double scal)
```

```
{
```

```

    double* res = new double[n];
    for (int i = 0; i < n; i++)
        res[i] = vec[i] * scal;
    return res;
}

double* VecOperations::AddVec(double* vec1, double* vec2)
{
    double* res = new double[n];
    for (int i = 0; i < n; i++)
        res[i] = vec1[i] + vec2[i];
    return res;
}

double VecOperations::DotProduct(double* vec1, double* vec2)
{
    double res = 0;
    for (int i = 0; i < n; i++)
        res += vec1[i] * vec2[i];
    return res;
}

```

5.9 VecOperations.h

```

#pragma once

class VecOperations
{
public:
    int n = 2;
    double* MultVecScal(double* vec, double scal);
    double* AddVec(double* vec1, double* vec2);
    double DotProduct(double* vec1, double* vec2);
};

```